Dynamic Portfolio Optimization in Equities and Prediction Markets

Naseebullah Andar Samantha Agisim Patrick Ledoit

Abstract

This paper studies dynamic portfolio optimization in two settings: a time-varying universe of U.S. equities and a daily panel of prediction-market contracts. In the equity application, portfolio weights are chosen by solving a long-only mean-variance problem with a turnover penalty, using rolling estimates of expected returns and covariance risk. The primary benchmark is an equal-weight portfolio formed on the exact same date-specific universe, so that performance differences can be attributed to the portfolio rule. The equity analysis shows that performance is highly sensitive to regularization, estimation design, and numerical calibration. Under a conservative baseline specification, the optimized portfolio behaves much like a smoothed version of equal weighting and delivers only modest gains. Once the same framework is recalibrated with lighter penalties and more effective solver settings, out-of-sample performance improves materially at both monthly and daily frequencies. The second part of the paper extends the framework to prediction markets, where prices represent market-implied probabilities rather than claims on cash flows. Using project artifacts built from Polymarket-style daily data, we construct a 60-day mispricing portfolio pipeline that maps contract-level features into estimated probabilities, approximates executable prices from the order book, and allocates capital with projected-gradient optimization under rolling covariance risk, cash, concentration, event, and liquidity constraints. The daily 60-day re-optimization run spans 343 decision dates from 2025-05-13 to 2026-04-20 and covers 275 unique contract IDs in the saved outputs. Relative to the static fold-based baseline, the daily implementation slightly lowers the mean objective (0.0505 versus 0.0525) while sharply reducing turnover (0.0767 versus 0.3778). However, realized next-step profit and loss, computed from subsequent quote changes because final resolutions are not included in the packaged artifact, remains fragile and is negative on train, validation, and test splits. The combined evidence suggests that the optimization framework is portable across asset classes, but also that calibration, execution assumptions, and evaluation design are integral parts of the effective portfolio rule. In equities, these choices determine whether optimization materially improves on naive diversification. In prediction markets, they determine whether apparent mispricing signals translate into stable economic value or remain primarily a research diagnostic.

1 Introduction

Portfolio choice under uncertainty is one of the central problems in financial economics. In principle, an investor who can estimate expected returns and risk accurately should be able to improve on naive diversification by allocating more capital to attractive assets and less to unattractive ones. In practice, however, portfolio optimization is fragile. Expected returns and covariance matrices must be estimated from finite samples, and small perturbations in those estimates can induce large changes in the resulting portfolio. This problem appears already in the classic mean-variance setting of Markowitz (1952), and it remains central in modern empirical work comparing optimized portfolios with simple benchmark rules such as equal weighting (DeMiguel, Garlappi, and Uppal, 2009).

This project studies portfolio allocation as a numerical optimization problem. The decision variable is a vector of portfolio weights, the objective balances expected return against risk and trading frictions, and the feasible set imposes economically meaningful restrictions through long-only, full-investment, or cash-allowing constraints depending on the application. This perspective is useful because it forces the researcher to specify the loss, encode the constraints, choose an algorithm, and then evaluate the realized behavior of that algorithm through diagnostics such as turnover, drawdown, cumulative performance, and sensitivity to tuning choices. The resulting strategy is not determined by the objective function alone. It also depends on how the investable universe is defined, how moments or signals are estimated, how strongly the problem is regularized, and how the numerical solver is tuned. Those elements are part of the effective portfolio rule.

The first application in the paper develops this idea in a dynamic universe of U.S. equities. At each date, the optimizer solves a rolling mean-variance problem with a turnover penalty under long-only simplex constraints. The strategy is implemented directly in PyTorch using a softmax parameterization of weights, which automatically satisfies nonnegativity and the adding-up constraint while allowing gradient-based optimization. The equity results show a clear pattern. A conservative baseline specification stays too close to equal weighting to produce substantial gains. Once the same framework is recalibrated with lighter risk and turnover penalties, a better learning rate, and a sufficient gradient-step budget, the optimized strategy materially outperforms the equal-weight benchmark in monthly and daily

backtests.

The second application extends the framework to prediction markets, following the broad idea that market prices can be interpreted as probabilistic forecasts of future outcomes (Wolfers and Zitzewitz, 2004; Arrow et al., 2008). In this setting, contracts do not pay continuous cash flows like equities. Instead, they resolve to event outcomes, so value comes from identifying mispriced probabilities and trading them under realistic execution and capital constraints. We use the project's Polymarket-style research pipeline as a baseline implementation. The pipeline starts from cleaned contract-level data, builds a ridge-based probability forecast, approximates executable purchase prices from the order book, and allocates capital with projected-gradient optimization subject to contract, event, and liquidity limits. The most important lesson from this extension is methodological: a positive optimization objective does not automatically imply stable realized profit and loss. Under the saved 60-day daily re-optimization run, the model continues to produce positive mean objective values, but realized next-step quote-to-quote PnL remains weak and negative across all splits.

The paper therefore makes two linked contributions. First, it documents that in the equity setting, dynamic portfolio optimization can outperform high-performing market indices once regularization, estimation design, and numerical calibration are treated as first-order modeling choices. Second, the paper shows that the same optimization framework can be transported to prediction markets, but with a different economic interpretation. Because prediction-market prices represent market-implied probabilities rather than claims on corporate cash flows, the key outputs are not only realized returns but also ranking strength, feasible allocation, turnover discipline, liquidity exposure, and chronology-safe quote-to-quote validation. The prediction-market extension therefore reframes portfolio optimization as both an allocation method and a diagnostic tool for testing whether estimated mispricing survives realistic constraints.

2 Data Sources

2.1 Equity return panels

The equity analysis uses two return datasets indexed by date and PERMNO. In both datasets, observations include stock returns together with standard identifying variables such as share code, exchange code, and, when available, price. The return variable is `ret`. When present, `shrcd` and `exchcd` are used to restrict the sample to common shares listed on major U.S. exchanges, and `prc` is used for an auxiliary low-price filter. In the monthly dataset, after cleaning and standard equity screens are applied, the sample contains 300 monthly dates and 971 distinct securities, producing a raw date-by-security panel of dimension 300×971. In the daily dataset, the filtered sample contains 6,289 trading dates and 978 distinct securities, producing a raw panel of dimension $6,289 \times 978$. The return panels are unbalanced. Securities do not appear continuously over the full sample, and the paper treats that variation in availability as an integral feature of the design rather than as an inconvenience to be suppressed mechanically. A stock enters the investable universe only after it accumulates sufficient return history for rolling estimation. This creates a time-varying opportunity set while ensuring that expected returns and covariance matrices are estimated from minimally informative historical windows. In addition to stock-level returns, the project uses adjusted price series for broad passive benchmarks such as the S&P 500, Nasdaq 100, and Russell 2000 to provide external economic context. These benchmark series are not used in portfolio construction itself; they are used only as comparators in the discussion of results.

2.2 Prediction-market artifact

The prediction-market extension begins with the cleaned Polymarket exploration pipeline documented in the project Markdown files. The exploration notebook starts from `polymarket_markets_rich.csv`, a 500×92 snapshot of open markets in the checked-in exploration artifact, and parses a set of fields that are directly useful for subsequent modeling: contract identifiers, question text, slugs, JSON outcome-price arrays, event JSON, volume and liquidity measures, order-book information such as `bestBid`, `bestAsk`, and `spread`, platform flags, and timing fields such as `createdAt` and `endDate`. The notebook also extracts the first entry of `outcomePrices` as an implied "Yes" price and parses event identifiers from the `events` JSON object. The final 60-day optimization pipeline uses the richer daily artifact embedded in the project outputs rather than the one-shot exploration snapshot alone. The saved daily re-optimization output spans 343 decision dates from 2025-05-13 through 2026-04-20 and covers 275 unique contract IDs. In the saved optimization output, there are on average 185.9 contract observations per decision date, with a median of 227. The object of interest is not final binary resolution because the packaged artifact does not include realized outcomes. Instead, the daily backtest evaluates quote-to-quote changes using the next observed implied price as the realized comparison point. This design is not a substitute for final contract resolution, but it is sufficient to test whether the model's estimated edge survives chronology-safe re-optimization and conservative execution assumptions.

3 Optimization Framework

3.1 Equity portfolio rule

The equity strategy is defined by four ingredients: a time-varying investment universe, rolling estimation of moments, a constrained portfolio objective, and a sequential backtest. At each portfolio formation date t, a stock is eligible only if it has accumulated sufficient prior return history. In the monthly specification, this minimum history requirement is 60 prior monthly returns. When price data are available, securities with absolute price below 0.1 are excluded. After the filters are applied, the eligible universe is truncated at 350 securities per date. Conditional on the active universe at date t, expected returns and covariance risk are estimated from a trailing window of historical returns. Let $R_{t-L:t-1}$ denote the matrix of past returns over the previous L periods. The rolling mean vector and covariance matrix are

$$\hat{\mu}_t = \frac{1}{L} \sum_{s=t-L}^{t-1} r_s, \qquad \hat{\Sigma}_t = \mathrm{Cov}(R_{t-L:t-1}).$$

In the monthly baseline, $L = 24$. Before optimization, only securities with a fully observed history over the estimation window are retained. The covariance estimate is symmetrized and regularized with a small ridge term,

$$\hat{\Sigma}_t \leftarrow \frac{1}{2}(\hat{\Sigma}_t + \hat{\Sigma}_t^\top) + \varepsilon I, \qquad \varepsilon = 10^{-6},$$

which improves numerical stability when the cross section is large relative to the rolling sample size. Portfolio weights are then chosen by solving the penalized mean-variance problem

$$\max_{w_t} \quad \hat{\mu}_t^\top w_t - \gamma\, w_t^\top \hat{\Sigma}_t w_t - \kappa \left\| w_t - w_{t-1} \right\|_1$$

subject to

$$w_t \geq 0, \qquad \mathbf{1}^\top w_t = 1.$$

The first term rewards expected return, the second penalizes variance, and the third penalizes turnover relative to the previous portfolio. The strategy is implemented with a softmax parameterization,

$$w_t = \mathrm{softmax}(v_t),$$

so the simplex constraint is enforced automatically. Optimization proceeds by gradient-based methods in PyTorch, starting each rebalance from the current allocation whenever a previous portfolio exists. Realized returns at $t+1$ are then used exclusively for out-of-sample evaluation,

$$R^p_{t+1} = w_t^\top r_{t+1}, \qquad R^{ew}_{t+1} = \frac{1}{N_t} \mathbf{1}^\top r_{t+1},$$

where R^{ew}_{t+1} is the equal-weight benchmark formed on the exact same active universe. This is the paper's internal benchmark because it isolates the value added by the optimization rule itself.

3.2 Prediction-market adaptation

The prediction-market extension retains the same general structure—signal estimation, constrained optimization, and walk-forward evaluation—but reinterprets the meaning of expected return. Here prices represent market-implied probabilities, so the central object is an estimated mispricing or edge relative to an executable purchase price. The saved baseline implementation uses a ridge model (Hoerl and Kennard, 1970) fit to the logit of the current implied probability. Features are standardized on the training sample only and include transformed volume, liquidity, spread, a competitiveness indicator, time to expiry, and an inverse-liquidity proxy. If $\hat{p}_{i,t}$ denotes the model-implied probability for contract i at date t, then the optimizer uses the edge

$$a_{i,t} = \hat{p}_{i,t} - c^{\mathrm{exec}}_{i,t},$$

where $c^{\mathrm{exec}}_{i,t}$ is a conservative executable "Yes" price approximation. In the project code, the execution rule uses `bestAsk` when available, otherwise the midquote, otherwise the implied price plus half the spread, and otherwise the implied price itself. At each decision date, the portfolio solves

Table 1: Parsimonious summary of the most important empirical results. Equity values summarize the reported monthly backtests; prediction-market values summarize the saved daily 60-day artifact.

Setting	Baseline/comparator	Recalibrated or operational run	Main interpretation
Equities: conservative monthly optimizer vs equal weight	Sharpe 0.6946 vs 0.6867; cumulative return 6.4286 vs 6.5265; turnover about 0.0064	With lighter penalties and stronger solver settings, Sharpe rises to 0.8204 and cumulative return to 28.1645 in the strongest exploratory monthly run	The baseline is over-regularized and behaves like smoothed equal weighting; calibration determines whether optimization is economically active.
Prediction markets: static fold run vs daily 60-day re-optimization	Static mean objective 0.0525; mean turnover 0.3778	Daily mean objective 0.0505; mean turnover 0.0767; test invested capital 0.941; test total next-step PnL -0.0083	Daily optimization is more operationally realistic and low-turnover, but quote-to-quote PnL remains negative.

$$\max_{w_t} \quad \sum_i w_{i,t}(\hat{p}_{i,t} - c_{i,t}^{\text{exec}}) - \gamma \sum_i w_{i,t}^2 \hat{p}_{i,t}(1-\hat{p}_{i,t}) - \kappa \|w_t - w_{t-1}\|_1$$

subject to

$$w_{i,t} \geq 0, \qquad \sum_i w_{i,t} \leq 1, \qquad w_{i,t} \leq \bar{w}, \qquad \sum_{i \in e} w_{i,t} \leq \bar{w}_e, \qquad \sum_i \ell_{i,t} w_{i,t} \leq B.$$

The prediction-market problem therefore allows cash and does not force full investment. In the saved baseline configuration, the project code sets $\bar{w} = 0.08$, $\bar{w}_e = 0.25$, and a liquidity budget $B = 2.5$, with liquidity loading $\ell_{i,t}$ proportional to inverse square-root depth. The optimizer is a projected subgradient method in the spirit of constrained first-order optimization (Duchi, Hazan, and Singer, 2011). After each gradient step, a fast cyclic projection enforces the total-weight, per-contract, per-event, and liquidity constraints. The daily walk-forward loop is chronology-safe by construction. Training uses only rows dated strictly before the decision date, feature scaling is fit on the training sample only, and realized evaluation for a contract is based on the next observed implied price in that contract's time series. For the saved outputs, the daily run uses a 60/20/20 split over decision dates, producing train, validation, and test summaries. The baseline hyperparameters are those stored in the project configuration: ridge penalty $\alpha = 5.0$, risk penalty $\gamma = 2.0$, turnover penalty $\kappa = 0.05$, and 800 projected-gradient iterations per re-optimization date. Because the packaged artifact does not include final contract resolution, the main evaluation metrics are mean objective, mean squared error between $\hat{p}$ and the current implied price, turnover, split-wise next-step PnL, and allocation diagnostics.

4 Equity Results

4.1 Conservative baseline

The equity analysis begins with a conservative baseline specification. Under that implementation, the optimized portfolio differs only modestly from the equal-weight benchmark formed on the same active universe. The annualized Sharpe ratio rises slightly, from 0.6867 for equal weight to 0.6946 for the optimized strategy, while annualized volatility falls from 0.1691 to 0.1654. Maximum drawdown improves marginally, from -0.5344 to -0.5297. These gains, however, do not come with stronger cumulative performance. The optimized portfolio records a mean monthly return of 0.0096 and cumulative return of 6.4286, compared with 0.0097 and 6.5265 for equal weight. Economically, the baseline optimizer behaves mainly as a smoothed version of equal weighting.

The weight dynamics explain this weak baseline result. Across the out-of-sample rebalancing dates, the strategy remains broadly diversified, the largest average weights stay below one-half of one percent, and average one-period turnover is only about 0.0064 in L^1 distance. This is attractive from an implementation standpoint, but it leaves little room for material outperformance. A portfolio that remains close to diversified equal weighting and trades only minimally is unlikely to generate large cumulative differences.

4.2 Recalibration and sensitivity

Diagnostic exercises show that the weak baseline result is not evidence against the underlying objective itself. In a simple two-asset diagnostic, the optimizer stays close to equal weights under baseline solver settings even when one asset has much higher expected return, but it tilts strongly toward the high-return asset once regularization is lightened and the numerical configuration is made more aggressive. This indicates that the conservative baseline is under-moving because of the interaction between strong regularization and limited convergence. The recalibrated monthly model performs much better. A moderately aggressive specification with $\gamma = 1.0$, $\kappa = 0.005$, learning rate 0.1, and 1000 gradient steps raises mean monthly return to 0.0125, annualized Sharpe to 0.7996, and cumulative return to 12.8233, albeit with somewhat higher volatility. A still lighter-penalty specification with $\gamma = 10^{-5}$ and $\kappa = 10^{-5}$ produces the strongest monthly backtest in the exploratory search: mean monthly return 0.0167, annualized Sharpe 0.8204, and cumulative return 28.1645. These results show that the same objective can behave either like a near-passive benchmark or like a materially active strategy depending on how it is regularized and numerically realized.

A systematic grid over risk aversion and turnover penalization reinforces this point. The strongest monthly results occur in the low-penalty region, with the best configuration in the reported grid at $\gamma = 0.001$ and $\kappa = 10^{-5}$. Nearby combinations perform almost as well, so the strong region is broad rather than knife-edge. Performance deteriorates steadily when either the risk penalty or turnover penalty becomes too large, because the optimizer is pushed back toward a benchmark-like allocation. Within that low-penalty region, solver hyperparameters are themselves economically consequential. The best learning-rate and step-budget combination in the exploratory monthly search is learning rate 0.1 with 1000 gradient steps. Very small learning rates leave the optimizer close to its initialization, while excessively large step budgets can degrade performance rather than improving it.

4.3 Rolling windows, external benchmarks, and daily extension

The exploratory search over rolling estimation windows is hump-shaped. Holding the preferred low-penalty configuration fixed, the best reported window is 24 months, which yields annualized Sharpe 0.8204 and cumulative return 28.1607. Neighboring windows of 18 and 36 months also perform very well, while shorter windows appear too noisy and longer windows appear too inertial. This pattern suggests that the main challenge is not simply to estimate means and covariances as accurately as possible in a statistical sense, but to summarize historical information at a horizon that balances responsiveness against noise. The strongest optimized monthly strategy also appears economically competitive with broad passive benchmarks. Over the 2004–2024 comparison window described in the earlier equity analysis, one dollar invested in the S&P 500 grows to just under eight dollars, one dollar in the Nasdaq 100 grows to more than sixteen dollars, and one dollar in the Russell 2000 grows to roughly five dollars. By contrast, the strongest optimized monthly specification reaches terminal wealth of approximately 29.16 dollars. These benchmark comparisons should be read cautiously because the start date and rebalancing structure are not perfectly identical, but they suggest that the strongest optimized strategies are not merely beating a weak internal comparator.

The same logic survives the move to daily data once the model is made frequency-consistent. In the properly rescaled daily specification, the rolling window becomes 504 trading days, the minimum history becomes 1260 trading days, the turnover penalty is scaled to daily units, and the same basic optimizer is run with a daily information set. Under that design, the optimized daily portfolio raises mean daily return from 0.000501 to 0.000713, annualized Sharpe from 0.6153 to 0.7099, and cumulative return from 7.1347 to 18.0155 relative to the equal-weight benchmark on the same active universe. The daily extension therefore strengthens the main interpretation of the equity results: when regularization, estimation horizon, and numerical implementation are chosen coherently, optimization can materially improve on naive diversification.

Taken together, the equity results support five conclusions. First, the conservative baseline is too heavily regularized and too numerically under-moving to generate large gains over equal weighting. Second, the weak baseline is not a failure of the objective itself; it is a consequence of the way that objective is tuned and solved. Third, the strongest monthly results arise in a region of light regularization. Fourth, rolling-window length and solver configuration are economically meaningful because they change the realized portfolio rule. Fifth, the main mechanism survives the transition from monthly to daily data when the design is rescaled appropriately.

5 Prediction-Market Extension: 60-Day Mispricing Portfolio

5.1 Research design and workflow

The prediction-market extension is best understood as a baseline research pipeline rather than as a production-ready trading system. The two project markdown files provide the design logic. The exploration document summarizes the cleaned schema, JSON parsing, and basic descriptive analysis of volume, liquidity, and implied prices. The pipeline document then turns those cleaned fields into a chronology-safe portfolio routine: feature preparation, a ridge-based probability model, executable price approximation, projected-gradient allocation, and diagnostics for weights, turnover, and realized next-step PnL.

The saved daily run spans 343 decision dates and 275 unique contract IDs, with an average of 185.9 contracts available per decision date. In the test segment alone, the strategy invests on average 0.941 of capital, holds about 52 positive-weight positions per date, and never exceeds the per-contract cap of 0.08; these are useful diagnostics because the pipeline is designed to allow cash and constrain concentration rather than to force full investment mechanically.

The daily optimizer was also updated to use a rolling empirical covariance penalty. At each decision date t, the implementation forms a 60-day history of implied prices using only rows with decision time before t. If $q_{i,s}$ is the implied price of contract i on historical date s, the covariance matrix is estimated from daily implied-price changes,

$$\widehat{\Sigma}_t = \widehat{\mathrm{Cov}}\left(\Delta q_{1,s}, \ldots, \Delta q_{n_t,s}\right), \qquad \Delta q_{i,s} = q_{i,s} - q_{i,s-1}.$$

The optimized smooth objective therefore changes from a diagonal Bernoulli risk penalty to

$$w_t^\top a_t - \gamma w_t^\top \widehat{\Sigma}_t w_t,$$

with the same turnover penalty and feasibility constraints as before. Invalid covariance diagonals fall back to $\hat{p}_{i,t}(1-\hat{p}_{i,t})$, non-finite off-diagonal entries are set to zero, and a small ridge term is added for numerical stability. In the saved covariance run, the mean covariance risk is 0.00115 and the diagonal Bernoulli-equivalent risk is 0.00115, so the update preserves the original risk scale while allowing cross-contract dependence.

5.2 Static fold-based baseline versus daily re-optimization

Table 3 compares the static fold-based 60-day run with the daily re-optimization run. The static run has slightly higher mean objective, 0.0525, versus 0.0505 for the daily run. It also has slightly higher mean MSE summary, 0.0227 versus 0.0217. But the most striking difference is turnover. The static fold-based summary has mean turnover 0.3778, while the daily walk-forward run has mean turnover only 0.0767.

This comparison matters because the daily run is closer to the way a portfolio would actually be updated in practice. It uses only earlier observations at each decision date, carries forward previous weights, and exposes the tradeoff between edge-seeking and trading discipline. In that sense, the lower-turnover daily run is the more informative benchmark even if its average objective value is slightly smaller.

5.3 Split-wise next-step PnL and chronology-safe evaluation

The most important empirical result in the prediction-market extension is that positive objective value does not translate automatically into positive realized next-step PnL. Table 4 reports the split-wise summary from the daily 60-day re-optimization run. Mean objective remains positive in every split, ranging from 0.0415 on test to 0.0557 on validation. Yet total next-step PnL is negative in every split as well: -0.0145 on training dates, -0.0073 on validation dates, and -0.0083 on test dates.

This result has a straightforward interpretation. The objective is computed from estimated edge after penalties and constraints, so it can remain positive even when the subsequent quote move does not validate the trade. The daily backtest therefore reveals a gap between signal-based desirability inside the optimizer and realized quote-to-quote economic value outside the optimizer. In a sense, this is precisely the purpose of chronology-safe evaluation: it prevents objective value from being confused with evidence of stable alpha.

5.4 Turnover path and allocation behavior

Turnover diagnostics show that the penalty term is doing real work. The average test-split turnover is 0.181, and overall mean turnover in the daily run is only 0.0767. Even so, the turnover path contains spikes. These episodes are important because they indicate dates when the optimizer makes unusually large reallocations in response to changes in the estimated edge set, liquidity conditions, or both. In an implementation setting, those spikes are model-risk events that would deserve separate investigation.

The weight-versus-edge relationship confirms that the optimizer is directionally sensible. Contracts with strongly negative estimated edge receive zero or near-zero weight, while contracts with positive estimated edge receive larger allocations. However, the mapping is clearly not one-for-one. The covariance risk penalty, turnover penalty, per-contract cap, event cap, and liquidity budget all compress extreme allocations. As a result, the strategy behaves like a disciplined selector of positively scored contracts rather than a raw bettor on whichever contracts have the highest model score.

The top-position chart in the saved artifact is best interpreted descriptively rather than economically. It identifies the contract IDs that receive the highest average test weights, but not their semantic labels, because the packaged summary outputs preserve contract identifiers rather than full contract text. Even so, it is useful as a concentration diagnostic: the top average test weights are in the 2–3 percent range, which is consistent with a portfolio that spreads exposure across many contracts rather than concentrating aggressively in a handful of names.

5.5 Interpretation

The main result of the prediction-market extension is not that the model generates a large stable trading profit; it does not. Instead, the result is that the optimization pipeline is coherent, reproducible, and informative. It creates a leakage-safe mapping from cleaned market data to estimated probabilities, feasible allocations, turnover paths, and realized quote-to-quote diagnostics. The daily re-optimization run is especially valuable because it surfaces the tension between three desiderata that are easy to conflate when looking only at a single backtest number: positive objective value, low turnover, and positive realized PnL. In the saved artifact, the first two are present, but the third is not yet robust.

6 Validation and Implementation Checks

The main analyses share a common validation discipline. At every decision date, the inputs used to form the portfolio are dated strictly before the realized outcome being scored. In the equity application this means that rolling means and covariances are computed from lagged returns and the benchmark is rebuilt on the same active universe. The comparison is therefore not a comparison between an optimized strategy and a different stock-selection universe; it is a comparison between two allocation rules applied to identical names at identical dates. This detail is important because universe drift can otherwise dominate the measured value of the optimizer. A change in performance should be interpreted as an allocation effect only after the timing and the universe have been held fixed.

A second check concerns the meaning of turnover. The equity baseline has very low turnover, which makes it implementable but also explains why it has limited scope to outperform. Recalibrated specifications trade more actively, so their higher returns should be assessed together with the implicit trading burden. In the prediction-market run, turnover is not merely a cost proxy; it is also an operational diagnostic. Spikes identify dates when the optimizer substantially changes its preferred event exposures. Those dates should be audited in any future production version because they may reflect genuine changes in edge, changes in liquidity, or model instability around a small subset of events.

A third check is numerical. The same mathematical objective can behave differently when solved with different parameterizations, learning rates, or step budgets. The equity implementation uses a softmax representation to remain on the long-only simplex, while the prediction-market implementation uses projected-gradient updates to respect cash, concentration, event, and liquidity constraints. These choices are not cosmetic. They shape the feasible path that the solver explores and determine whether the optimizer is effectively passive or active. The reported recalibration results should therefore be read as evidence about a complete implemented rule rather than about an abstract mean-variance formula in isolation.

Finally, the prediction-market evaluation separates internal objective value from external realized validation. The objective rewards estimated edge net of risk and trading penalties, but the realized outcome is the subsequent quote movement available in the artifact. Because final binary resolutions are missing, this realized measure cannot be interpreted as settlement PnL. It is still useful because it asks whether the contracts that look attractive at decision time move favorably at the next observation. In the saved 60-day run they do not, on average. That negative result is a strength of the design: it prevents a well-behaved optimizer from being mistaken for a profitable signal.

7 Discussion and Conclusion

This project studies portfolio optimization as an empirical pipeline rather than as a standalone mathematical objective. In both applications, the portfolio rule is created jointly by the data construction, the estimation procedure, the regularization choices, the constraints, and the numerical solver. This point is especially visible in the equity analysis. Under a conservative baseline, the optimizer trades very little, and produces only modest improvements in risk-adjusted performance. That result might initially suggest that optimization adds little value beyond naive diversification. The subsequent diagnostic and recalibration exercises show a different interpretation. Once the risk penalty, turnover penalty, learning rate, gradient-step budget, and rolling estimation window are adjusted coherently, the same basic optimization framework produces much stronger out-of-sample performance in the sequential rolling-window backtests. The equity results therefore support a practical lesson about dynamic portfolio choice. The mean-variance objective is not enough. A portfolio optimizer can behave like a nearly passive benchmark if it is over-regularized or insufficiently optimized, even when the underlying objective allows active tilts. Conversely, when penalties and solver settings permit the optimizer to respond to estimated return variation, the strategy can materially improve on the equal-weight benchmark formed on the same date-specific universe. The daily extension strengthens this conclusion because the same logic continues to work after the estimation window, minimum-history requirement, and turnover penalty are rescaled to the higher-frequency setting. Across these exercises, the central result is that coherent calibration is economically meaningful. The prediction-market extension shows both the portability and the limits of this framework. The same basic structure can be carried from equities into Polymarket-style contracts: estimate a signal, approximate executable prices, impose capital and concentration constraints, penalize rolling covariance risk and turnover, and evaluate performance in a walk-forward design.

However, the interpretation changes because prediction-market contracts are probabilistic claims rather than equity securities. A positive model-implied edge does not guarantee a profitable subsequent quote movement, and a positive optimization objective does not automatically imply realized economic value. In the saved 60-day daily re-optimization run, the optimizer produces disciplined allocations, keeps turnover low, and assigns weight primarily to contracts with positive estimated edge. At the same time, realized next-step quote-to-quote PnL remains negative across train, validation, and test splits. This gap between objective value and realized PnL is one of the most important findings of the extension. Several limitations qualify the interpretation of the results. The equity analysis uses rolling-window estimation, so each portfolio is formed using only information available before the realized return being evaluated. The equity results are therefore out of sample in the relevant sequential sense. The remaining caution is that the final hyperparameter choices were developed through exploratory calibration over the course of the project. For that reason, the strongest specifications should be interpreted as evidence that dynamic optimization can materially improve performance under coherent calibration, rather than as a finalized portfolio rule. In the prediction-market extension, the main limitation is that the packaged artifact does not include final contract resolutions. Realized PnL is therefore measured using next-step implied prices rather than terminal binary outcomes. This measure is useful for studying quote-to-quote signal persistence, but it is not equivalent to settlement-based trading profit. The prediction-market model also remains intentionally simple, leaving room for richer forecasting models. The broader conclusion is that portfolio optimization should be judged by the behavior of the full system, not by the elegance of the objective function alone. In equities, a carefully calibrated rolling-window optimizer can move beyond naive diversification and generate economically meaningful performance improvements. In prediction markets, the same framework provides a structured way to connect probability forecasts to feasible allocations and honest walk-forward diagnostics, even when the current baseline does not yet produce robust realized PnL. The project therefore points toward a unified view of optimization across markets: the value of a strategy depends not only on the signal it estimates, but on whether that signal survives the constraints, frictions, numerical choices, and evaluation design required to turn it into a portfolio.

References

Arrow, K. J., Forsythe, R., Gorham, M., Hahn, R., Hanson, R., Ledyard, J. O., Levmore, S., Litan, R., Milgrom, P., Nelson, F. D., Neumann, G. R., Ottaviani, M., Schelling, T. C., Shiller, R. J., Smith, V. L., Snowberg, E., Sunstein, C. R., Tetlock, P. C., Tetlock, P. E., Varian, H. R., Wolfers, J., and Zitzewitz, E. (2008). The Promise of Prediction Markets. *Science*, 320(5878), 877–878.

Bergstra, J. and Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research*, 13, 281–305.

DeMiguel, V., Garlappi, L., and Uppal, R. (2009). Optimal Versus Naive Diversification: How Inefficient Is the 1/N Portfolio Strategy? *The Review of Financial Studies*, 22(5), 1915–1953.

Duchi, J., Hazan, E., and Singer, Y. (2011). Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. *Journal of Machine Learning Research*, 12, 2121–2159.

Hoerl, A. E. and Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55–67.

Kingma, D. P. and Ba, J. (2015). Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*.

Markowitz, H. (1952). Portfolio Selection. *The Journal of Finance*, 7(1), 77–91.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, 32.

Shallue, C. J., Lee, J., Antognini, J., Sohl-Dickstein, J., Frostig, R., and Dahl, G. E. (2019). Measuring the Effects of Data Parallelism on Neural Network Training. *Journal of Machine Learning Research*, 20(112), 1–49.

Wolfers, J. and Zitzewitz, E. (2004). Prediction Markets. *Journal of Economic Perspectives*, 18(2), 107–126.

A Supplementary Tables and Figures

The appendix contains supporting tables and diagnostic figures. These pages are outside the eight-page main-paper count.

Table 2: Selected allocation diagnostics for the daily 60-day test segment.

Statistic	Value
Decision periods	70
Unique contracts in artifact	275
Average contracts available per period	185.9
Average positive-weight positions per period	52.0
Average invested capital	0.941
Maximum single-contract weight	0.08

Table 3: Comparison of the static fold-based 60-day run and the daily 60-day re-optimization run.

Run	Mean objective	Mean MSE	Mean turnover
Static fold-based 60-day run	0.0525	0.0227	0.3778
Daily 60-day re-optimization	0.0505	0.0217	0.0767

The daily run preserves chronology and recomputes weights each decision date using only earlier observations. The static fold-based run is useful as a baseline, but the daily run is closer to an operational deployment loop.

Table 4: Daily 60-day re-optimization summary by split.

Split	Periods	Total next-step PnL	Mean next-step PnL	Mean objective	Mean turnover
Train	202	-0.0145	-0.0001	0.0518	0.0672
Validation	71	-0.0073	-0.0001	0.0557	0.0009
Test	70	-0.0083	-0.0001	0.0415	0.1810

Realized PnL is computed from the next observed implied price, not from final contract resolution, because the packaged 60-day artifact does not include realized outcomes.

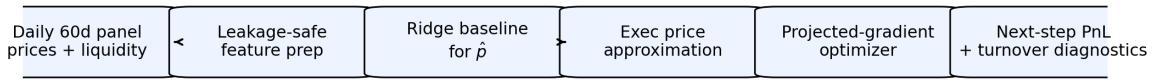
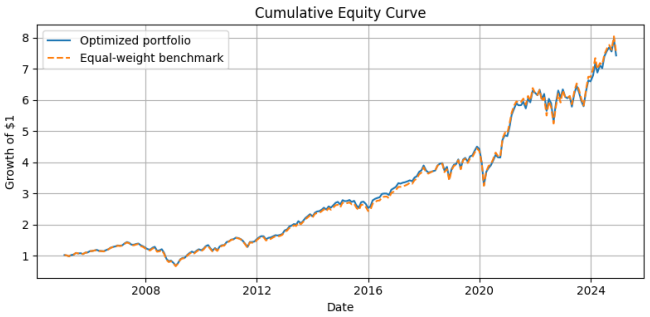
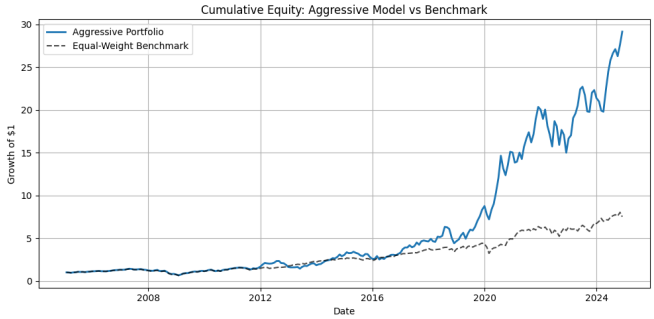


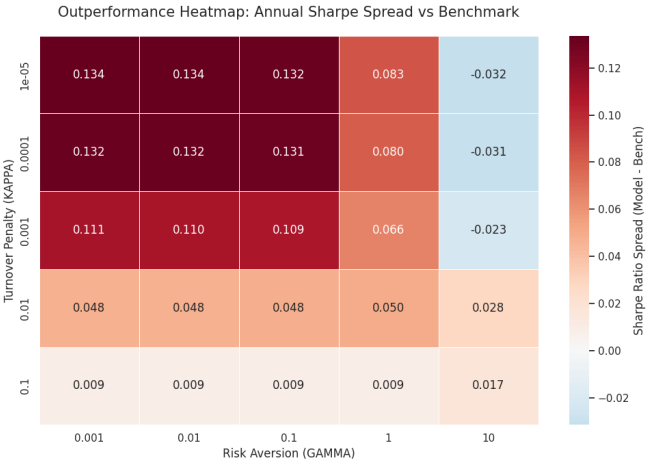
Figure 1: Prediction-market research pipeline for the 60-day daily experiment.



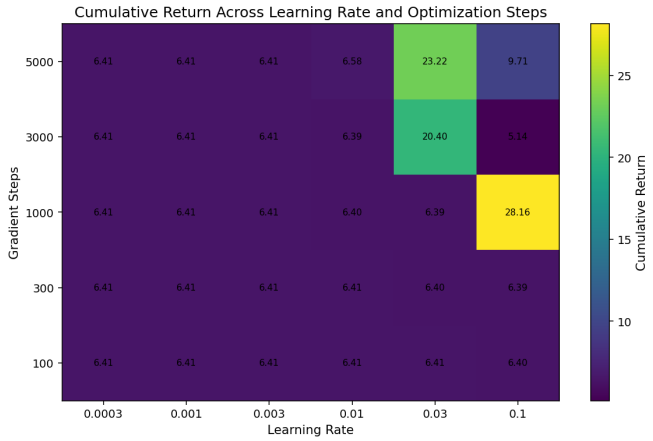
(a) Conservative monthly baseline versus equal weight.



(b) Recalibrated monthly performance.

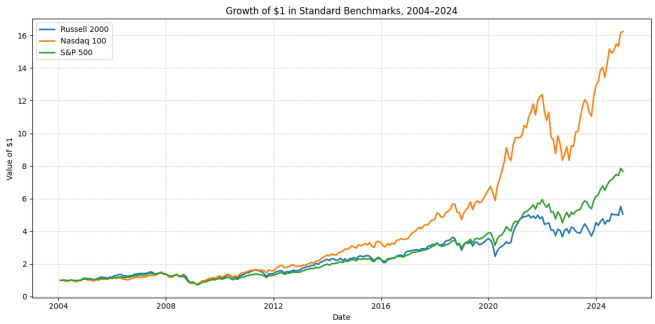


(c) Regularization and solver sensitivity.

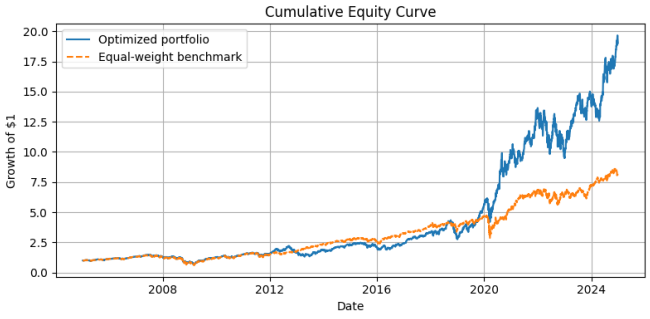


(d) Rolling-window sensitivity.

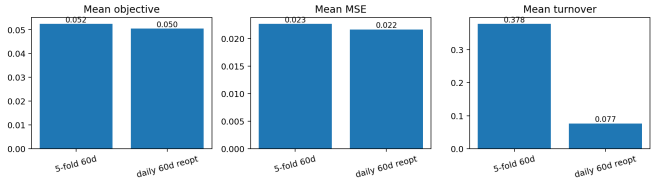
Figure 2: Equity diagnostics.



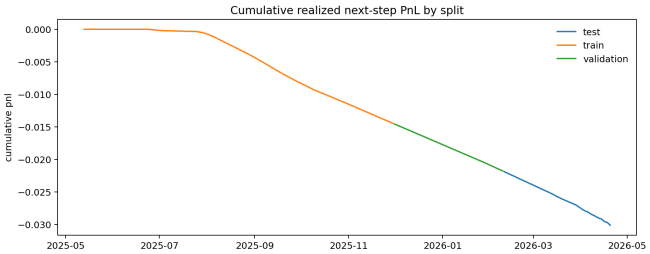
(a) External benchmark comparison.



(b) Daily-frequency equity extension.

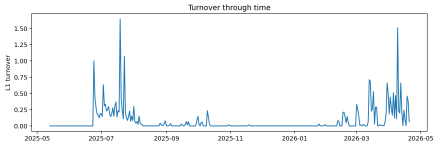


(c) Static versus daily prediction-market runs.

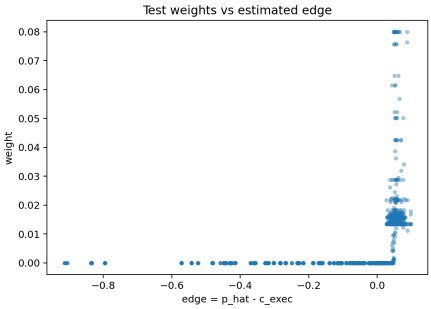


(d) Cumulative next-step PnL by split.

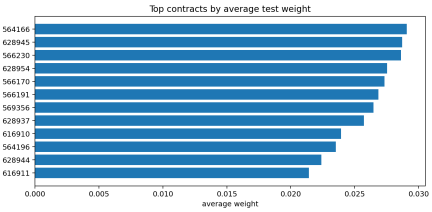
Figure 3: Benchmark and prediction-market PnL diagnostics.



(a) Turnover path.



(b) Weights versus estimated edge.



(c) Top test positions.

	n periods	total pnl	mean pnl	sharpe	turnover
test	70.0	-0.0083	-0.0001	-55.0661	0.181
train	202.0	-0.0145	-0.0001	-21.2294	0.0672
validation	71.0	-0.0073	-0.0001	-663.787	0.0009

(d) Split summary.

Figure 4: Prediction-market allocation diagnostics.